

Core Concepts — The Modern Python Toolchain

Track 16.1 — 🐍 Python for AI Engineering · 16-ai-tech-stack/tracks/python/notes/01-modern-python-toolchain.md

Scope: how you set up, type, structure, configure and test a Python AI project in 2026. Runtime performance lives in 02-numerics-async-and-performance.md.

1. Why Python won, and what that costs you

Python is not the fastest language, the safest, or the best designed. It won AI because it is the least bad **glue**: every fast thing — BLAS, CUDA, Arrow, tokenizers — is C, C++, Fortran or Rust with a Python binding, and NumPy's array model ("n-dimensional typed buffer with a shape") became the lingua franca that PyTorch, JAX, polars and every vector DB now speak. Network effects did the rest; every model ships Python examples first, sometimes only.

The bill: slow interpreter, historically disastrous packaging, type errors at runtime instead of compile time. The modern toolchain pays down the last two. The first you route around — note 02's subject.

2. uv — stop thinking about environments

uv (Astral, 2024, written in Rust) replaced the `pip + venv + pip-tools + poetry + pyenv` stack for most people inside about eighteen months, because it is 10–100× faster and does all five jobs with one binary. That speed is not a vanity metric: a 90-second CI install becoming 3 seconds changes how often you run CI.

```
curl -Lsf https://astral.sh/uv/install.sh | sh    # install (macOS/Linux)

uv init my-ai-service && cd my-ai-service      # pyproject.toml + .python-version + src/ layout
uv python install 3.13                          # uv manages interpreters too; no pyenv
uv add anthropic "pydantic>=2.9" httpx         # resolve + install + record + update uv.lock
uv add --dev pytest pytest-asyncio ruff mpy     # make the env exactly match uv.lock
uv sync                                         # run in the env without activating anything
uv lock --upgrade-package anthropic            # bump one dependency deliberately
uv tree                                         # who pulled in that transitive dependency?
```

Three things worth internalising:

- **uv run means you never source .venv/bin/activate again.** Activation is stateful, invisible in shell history, and the cause of half of all "works on my machine".
- **uv.lock is committed; .venv/ is not.** It pins every transitive dependency with hashes for every platform. `pip freeze > requirements.txt` is not reproducibility: it records *your* machine's resolution.

- **uv pip ... is an escape hatch** — drop-in pip for existing workflows (this repo uses it), but it does not manage the lockfile. In a new project, use `uv add`.

When to not use uv: you are inside a Conda ecosystem with non-Python binary dependencies (CUDA builds, geospatial, some HPC stacks) that only Conda resolves. Then use `pixi` or `conda` and move on. Everywhere else, uv.

The tool landscape, briefly

Tool	Still use it?	Verdict
uv	Yes	The default. Fast, single binary, manages Python versions too.
pip + venv	Only if uv is unavailable	Works. Slow. Doesn't lock.
poetry	Legacy projects	Fine, slower, its own resolver quirks. Don't start here.
pipenv	No	Effectively dead.
conda / mamba	Binary-heavy scientific stacks	The only real reason left.
pixi	Yes, if conda-flavoured	Conda packages with a modern lockfile and uv-like speed.

3. pyproject.toml — one file, whole project

`setup.py`, `setup.cfg`, `requirements.txt`, `.flake8`, `pytest.ini`, `mypy.ini`, `tox.ini` — all of it collapses into one declarative file (PEP 518/621). If a tool cannot be configured here, that is a signal about the tool.

```
[project]
name = "ticket-triage"
requires-python = ">=3.12"
dependencies = ["anthropic>=0.40", "pydantic>=2.9", "httpx>=0.27"]

[project.scripts]
triage = "ticket_triage.cli:main"          # installs a `triage` command on the PATH

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "SIM", "ASYNC", "RUF"]
ignore = ["E501"]                          # the formatter handles line length

[tool.mypy]
strict = true
[[tool.mypy.overrides]]
module = ["some_untyped_lib.*"]            # ML libraries often ship no stubs
ignore_missing_imports = true

[tool.pytest.ini_options]
addopts = "-q --strict-markers -m 'not live'"
asyncio_mode = "auto"
markers = ["live: hits a real API; skipped by default"]
```

Pin a lower bound, not an exact version, in `dependencies`. Exact pins belong in `uv.lock`. Pinning `==` here makes your library uninstallable alongside anything else.

4. `ruff` — lint and format, in one, instantly

Ruff (also Astral, also Rust) replaces flake8, isort, pydocstyle, pyupgrade, autoflake, and — since it grew a formatter — Black. It runs over a large codebase in well under a second, which is the point: a linter you run on every save changes behaviour; a linter you run in CI only generates arguments in code review.

```
uv run ruff check --fix .      # lint, autofix what is safely fixable
uv run ruff format .          # Black-compatible formatting
uv run ruff check --watch .    # live, while you work
```

Rule sets worth enabling beyond the defaults, and why: `I` (import sorting, removes an entire class of merge conflict) · `B` (bugbear — catches the mutable-default-argument bug from note 02 §9) · `UP` (pyupgrade — rewrites `Dict[str, int]` to `dict[str, int]`) · `SIM` (simplification) · `ASYNC` (flags blocking calls inside async functions — genuinely valuable for LLM code) · `RUF` (ruff's own, including mutable class attribute detection).

Do not argue about formatting. Run the formatter in a pre-commit hook and reclaim the meeting.

5. Types: what they buy and what they do not

Python type hints are **not enforced at runtime**. This is the single most misunderstood thing about them:

```
def embed(text: str) -> list[float]: ...
embed(42)                # runs. CPython does not check. The hint is metadata.
```

A hint is a message to three audiences: a type checker, your editor's autocomplete, and the next human. That is worth a great deal and is not the same as safety. Runtime safety comes from Pydantic (§6).

The checkers

	<code>mypy</code>	<code>pyright</code> / <code>Pylance</code>
Written in	Python	TypeScript
Speed	slower	fast, incremental
Editor integration	via LSP	native in VS Code (Pylance)
Strictness	very configurable	stricter defaults, better inference
Verdict	the CI gate most projects use	the editor experience most people already have

Running both is normal: `pyright` as you type, `mypy` in CI. Start `strict = true` on a new project. On an existing one, do **not** type everything at once — ratchet strictness per module with `[[tool.mypy.overrides]]` and `disallow_untyped_defs = true`.

The annotations that earn their keep in AI code

```
from typing import Literal, TypedDict, NotRequired, Protocol, Self

Role = Literal["user", "assistant", "system"]    # a closed set the checker enforces

class Message(TypedDict):                        # a dict with a known shape.
    role: Role                                  # zero runtime cost; it IS a dict.
    content: str                                # Perfect for API payloads.
    name: NotRequired[str]

class LLMClient(Protocol):                       # structural typing: anything with this
    async def complete(self, prompt: str) -> str: # method satisfies it. No inheritance,
    ...                                           # so your fake client type-checks too.
```

`Protocol` is the one to learn if you learn one. It is how you type "an LLM client" without coupling your code to a vendor SDK, and it is what makes the mock in your tests a first-class citizen rather than a `MagicMock` your checker knows nothing about.

TypedDict vs dataclass vs Pydantic — pick correctly

	TypedDict	@dataclass	Pydantic BaseModel
Runtime cost	none (it is a dict)	low	moderate (Rust core)
Validates input	no	no	yes
Serialises to JSON	already is	<code>asdict()</code>	<code>model_dump_json()</code>
Generates JSON Schema	no	no	yes
Use for	API request/response payloads	internal structs, configs, state	anything crossing a trust boundary

The decision rule: **data you created** → **dataclass**; **data someone else created** → **Pydantic**. LLM output is data someone else created, and that someone is a stochastic text generator.

6. Pydantic v2 — the contract at every boundary

Pydantic v2 rewrote its core in Rust (`pydantic-core`), making it fast enough to sit in a request path. For AI engineering it does three jobs from one class definition:

```
from pydantic import BaseModel, Field

class Citation(BaseModel):
    """A claim with the source sentence that supports it."""
    quote: str = Field(description="Verbatim sentence from the source document.")
    claim: str = Field(description="The claim that sentence supports.")
    confidence: float = Field(ge=0, le=1)

Citation.model_json_schema()    # -> the tool / structured-output schema you SEND
Citation.model_validate(payload) # -> validation of what you GET BACK
citation.claim                  # -> a typed attribute your checker understands
```

One definition, so schema and parser cannot drift. That is the direct link to Module 06 (tool calling) and Module 07 (agents): a tool definition is a JSON Schema, and the model's `tool_use` input is untrusted JSON.

Field order matters more than people expect. The model generates fields in schema order, so putting `quote` before `claim` makes it find its evidence before committing to a conclusion — chain-of-thought smuggled into a schema, worth real accuracy on extraction tasks.

`code/pydantic_structured_output.py` [16-ai-tech-stack/tracks/python/code/pydantic_structured_output.py] runs this end to end, including the repair loop, against simulated model responses.

7. Project layout for an AI service

```
ticket-triage/
├── pyproject.toml      # deps, tool config, entry points
├── uv.lock            # committed. reproducibility lives here
├── .python-version    # 3.13
├── .env.example       # KEY NAMES ONLY, no values. Committed.
├── .env               # real values. GITIGNORED. Never committed.
├── src/ticket_triage/
│   ├── config.py      # pydantic-settings; the ONLY place that reads os.environ
│   ├── schemas.py     # Pydantic models – the data contracts
│   ├── llm.py         # provider client + retry/rate-limit policy
│   ├── prompts/       # prompts as .md/.jinja files, not string literals
│   ├── pipeline.py    # business logic. Takes an LLMClient Protocol.
│   └── cli.py
└── tests/
    ├── conftest.py    # fixtures, including the fake LLM client
    ├── test_pipeline.py # mocked LLM, no network
    └── test_live.py   # @pytest.mark.live – opt in explicitly
```

Why `src/` layout: without it, `import ticket_triage` resolves to the source directory even when the package is not installed, so your tests pass locally and fail for everyone else. `src/` forces an install, which means your tests exercise what you actually ship.

Prompts belong in files, not in source. They change on a different cadence than code, they need diffing and review by non-engineers, and inlining a 900-token system prompt as a triple-quoted string makes the module unreadable. Load them with `importlib.resources`, version them, and log which version produced a given output.

8. Configuration and secrets

```
# config.py – the ONLY module that reads the environment, and it reads it once
from functools import lru_cache
from pydantic import Field, SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_prefix="TRIAGE_")
    anthropic_api_key: SecretStr # no default => required
    model: str = "claude-sonnet-4-5"
    max_concurrency: int = Field(default=8, ge=1, le=64)
    request_timeout_s: float = 120.0

@lru_cache
def settings() -> Settings:
    return Settings() # raises at startup if anything is missing or malformed
```

The value here is **fail-fast**. `os.environ["ANTHROPIC_API_KEY"]` blows up on the first request that needs it, which in practice is 3am on a Saturday. `Settings()` blows up in the first second of the process, in CI, in your smoke test — where a missing key is a five-second fix.

Secret hygiene, non-negotiable:

- `.env` is gitignored; `.env.example` lists key **names** with empty values and is committed.
- Production injects real environment variables (or a secret manager); it does not ship a `.env`.
- Never `print()` or log a config object. Use `SecretStr` so `repr()` renders `*****`, and audit your exception handlers — a traceback that includes `**kwargs` can leak a key into your log aggregator.
- Rotate keys on exposure, not on suspicion of exposure. See `../../../../13-ai-security/`.

9. Logging, not printing

`print()` writes an unstructured string to stdout with no level, no timestamp, no context, and no way to turn it off. It is fine in the three scripts in this track — they are teaching tools whose entire output is the lesson — and wrong in anything that runs unattended.

```
import logging, sys
logger = logging.getLogger(__name__) # module-level, __name__, never the root logger

logger.info("llm_call", extra={"model": model, "tokens_in": n_in, "latency_ms": ms})
```

Configure once, at the entry point only. Libraries configure nothing; they get a logger and log to it.

What to log in an LLM system, specifically:

Log this	Because
model id + prompt version	the two things that change your output without a code deploy
input/output token counts	cost is a first-class metric, and it is invisible otherwise
latency, split by attempt	p99 matters; a retry hides inside a "slow" request
retry count and terminal status	rising retries is the earliest signal a provider is degrading
a request/trace id, propagated	one user question can be twenty model calls
validation failures with the raw text	you cannot debug a parse failure from a stack trace

Use `structlog` or `stdlib` JSON formatting so logs are queryable. And do not log full prompts or completions by default — they contain user data, and that is a governance question (<https://openai.com/blog/12-ai-governance/>), not a convenience one.

10. Testing AI code

The unit of doubt is different here. Your code is deterministic; the model is not. Test them separately.

```
# conftest.py
class FakeLLM:
    """Satisfies the LLMClient Protocol. Scripted, instant, free, offline."""
    def __init__(self, responses: list[str]) -> None:
        self.responses, self.calls = list(responses), []
    async def complete(self, prompt: str) -> str:
        self.calls.append(prompt)
        return self.responses.pop(0)

@pytest.fixture
def fake_llm() -> FakeLLM:
    return FakeLLM(['{"sentiment": "negative", "urgency": 4}'])
```

The rules that matter:

1. **No network in the default run.** Mark live tests `@pytest.mark.live`, deselect with `addopts = "-m 'not live'"`. A suite that needs an API key is a suite nobody runs.
2. **Mock at the client boundary, not the HTTP layer.** Patching `httpx` couples your tests to a vendor's wire format; a fake satisfying your `Protocol` does not.
3. **Test parsing against *bad* output.** The clean-response test passes on day one and never catches anything again. Truncated JSON, fenced JSON, hallucinated enums, empty responses — your production incidents make excellent fixtures.
4. **Seed everything**, and know that `temperature=0` is *not* determinism: batching and floating-point nondeterminism on the provider side mean identical requests can still differ.
5. **Assert tolerances and properties, not exact strings.** `assert 0.6 < score < 0.9` survives a model version bump; `assert output == "..."` is a flaky-test generator.

6. **Snapshot the rendered prompt, not the completion** (`syropy`, `pytest-regressions`). A prompt diff in a PR is the review you actually want.

7. **Keep it under five seconds**, or people stop running it before pushing.

Coverage is a floor, not a goal. 90% coverage of happy paths in an LLM app tells you almost nothing.

11. Where this returns

Idea here	Where it returns
Pydantic schema -> tool definition	<code>.././././././06-ai-agents/</code> — tool calling
Validation + repair loop	<code>.././././././07-agentic-ai/</code> — agents that must not derail
Protocol-typed LLM client	<code>../langchain/</code> [16-ai-tech-stack/tracks/langchain] — why LangChain's interfaces look like this
<code>uv</code> + <code>pyproject.toml</code>	<code>../claude-code/</code> [16-ai-tech-stack/tracks/claude-code] and <code>.././././././09-mcp/</code> — MCP servers are packaged Python
Secrets, log hygiene	<code>.././././././13-ai-security/</code> , <code>.././././././12-ai-governance/</code>
Tolerance-based assertions, seeds	<code>.././././././15-ai-evals/</code> — the same problem at suite scale
Typed config, fail-fast startup	<code>.././././././10-ai-architecture/</code> — service design
TypedDict / Protocol	<code>../typescript/</code> [16-ai-tech-stack/tracks/typescript] — the same ideas, enforced by a compiler

Next: 02-numerics-async-and-performance.md — making it fast, and the idioms that will bite you on the way.